

In this exercise, you'll enhance your AI chatbot by adding image generation capabilities using Amazon Bedrock's foundation models. Your chatbot will be able to create and display images based on user text prompts, making it a truly multi-modal AI assistant.

What You'll Learn

By completing this exercise, you will:

- **Integrate** Amazon Bedrock's image generation models
- **Implement** multi-modal AI capabilities in your chatbot
- **Handle** image encoding and direct delivery through base64
- **Experience** advanced AI feature development with Kiro

Step 1: Add Image Generation Feature

Instruct Kiro to enhance your chatbot with image generation functionality:

Now I would like to add the ability to generate images in the chatbot. To simplify this task, we can look at the last request from the user. If the last request starts with "create image",

To keep it simple, convert the generated image to base64 in the Lambda function and send it directly to the UI. The frontend can display the base64 image without needing to store it in S3.

Please update both the backend and frontend to support this feature.

What Kiro Will Update

Your enhancement will include:

- **Backend Lambda:** Enhanced with image generation logic and Bedrock integration
- **Base64 Conversion:** Convert generated images to base64 format in Lambda
- **IAM Permissions:** Updated roles for Bedrock image generation access
- **Frontend Interface:** Updated chat UI to display base64 images
- **Direct Delivery:** Images sent directly in API response without external storage

✔ **Advanced AI Integration**

Notice how this builds on your existing AI infrastructure, adding new capabilities without disrupting the current text-based functionality.

Step 2: Test Image Generation

Test your new image generation capability:

Please test the image generation by trying both a normal text message and an image generation request like "create image a sunset over mountains". Show me that both work.

Expected Behavior

Your enhanced chatbot should demonstrate:

- ▶ **Text Message Example**
- ▶ **Image Generation Example**

Step 3: Verify Everything Works

Make sure your implementation works correctly:

Please verify that images are converted to base64 properly and display correctly in the chat interface. Make sure both text responses and image generation work together.

Success Criteria

After completing this exercise, you should have:

- ✔ **Image Generation:** Chatbot can create images from text prompts
- ✔ **Base64 Encoding:** Images converted to base64 format in Lambda
- ✔ **Direct Delivery:** Images sent directly in API response
- ✔ **Enhanced UI:** Chat interface displays both text and images
- ✔ **Dual Functionality:** Both text responses and image generation work properly

✔ **Multi-Modal AI Complete**

Your chatbot is now a sophisticated multi-modal AI assistant capable of both intelligent conversations and creative image generation!

Troubleshooting

Common issues you may encounter and their solutions:

- ▶ **Base64 Size Limitations**
- ▶ **Base64 Encoding Issues**
- ▶ **Bedrock Model Access Issues**
- ▶ **Image Display Issues**

Architecture Overview

- ▶ **View Enhanced Multi-Modal Architecture**

This exercise demonstrates how to extend AI applications with advanced capabilities while maintaining security and architectural best practices.

Key Takeaways

This exercise demonstrated advanced AI integration concepts:

Technical Skills

AI Development

Problem-Solving

- **Multi-Modal AI:** Integrated both text and image generation capabilities
- **Conditional Logic:** Implemented smart routing based on user input
- **Base64 Encoding:** Used direct image encoding for simplified delivery
- **Enhanced UI:** Updated frontend to handle multiple content types
- **Advanced Permissions:** Configured IAM for multiple Bedrock services

Next Steps

Your chatbot now combines intelligent conversation with creative image generation, making it a truly advanced multi-modal AI assistant. The next exercise will add even more exciting capabilities to enhance the user experience further.

Workshop Progress

